광운대학교 로봇게임단 로빛

로봇팔 IK

로봇팔 IK 계산 및 시연영상

학 번	2024402035
이 름	장경민
제출일	2026. 11. 13.

목차

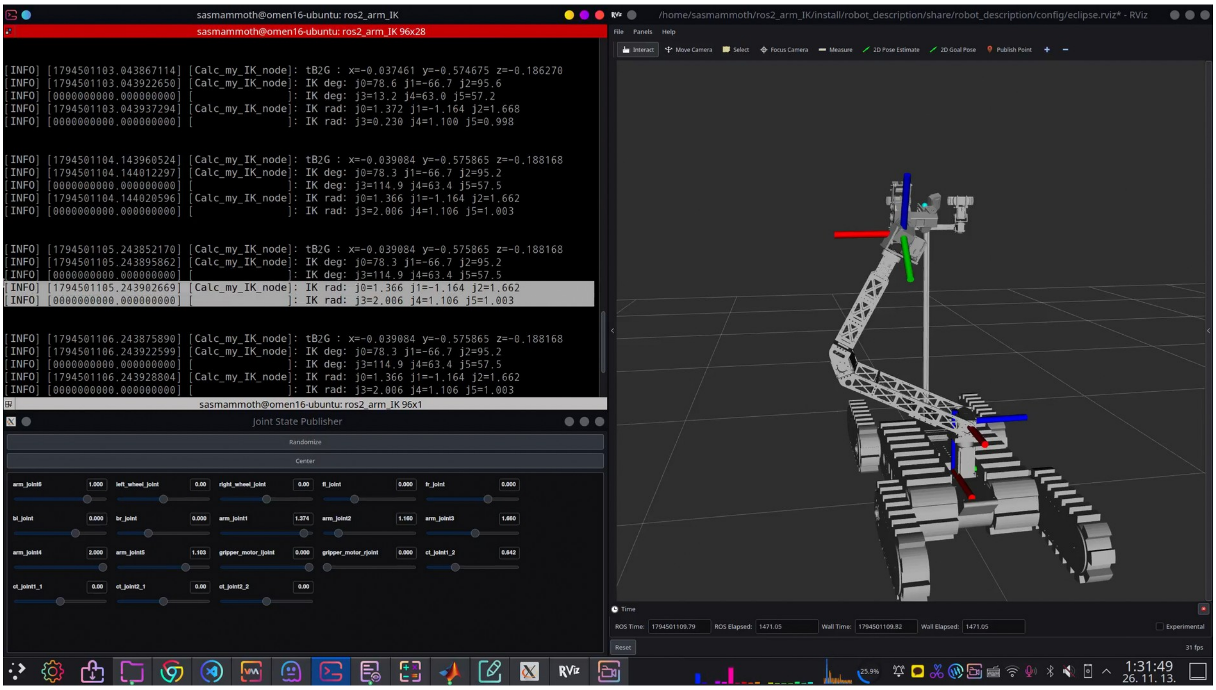
1. 시연영상..2

2. IK 계산 및 기타..4

1. 시연영상

영상 1 : https://drive.google.com/file/d/1D40dTOSOpJlZ3bkrvamZ7VjZ5dl241l8/view?usp=drive_link

영상 2 : https://drive.google.com/file/d/103uTzzpLuTmQ4kv6Q119AQzmacwNeZOr/view?usp=drive_link


sasmammoth@omen16-ubuntu: ros2_arm_IK
sasmammoth@omen16-ubuntu: ros2_arm_IK 96x28
sasmammoth@omen16-ubuntu: ros2_arm_IK 96x1
Joint State Publisher
Randomize
Center
/home/sasmammoth/ros2_arm_IK/install/robot_description/share/robot_description/config/eclipse.rviz* - RViz
File
Panels
Help
Interact
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
2D Goal Pose
Publish Point
Time
Reset
31 fps
1:31:49
26. 11. 13.

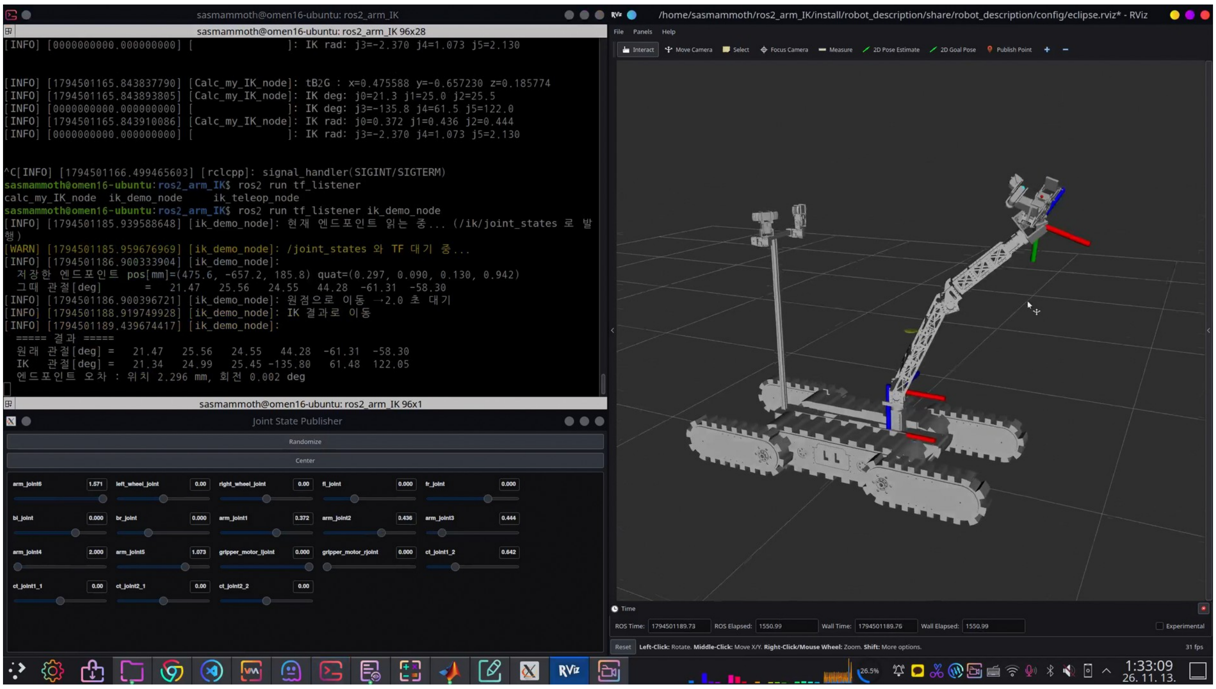
```
[INFO] [1794501103.043867114] [Calc_my_IK_node]: tB2G : x=-0.037461 y=-0.574675 z=-0.186270
[INFO] [1794501103.043922650] [Calc_my_IK_node]: IK deg: j0=78.6 j1=-66.7 j2=95.6
[INFO] [0000000000.000000000] [               ]: IK deg: j3=13.2 j4=63.0 j5=57.2
[INFO] [1794501103.043937294] [Calc_my_IK_node]: IK rad: j0=1.372 j1=-1.164 j2=1.668
[INFO] [0000000000.000000000] [               ]: IK rad: j3=0.230 j4=1.100 j5=0.998


[INFO] [1794501104.143960524] [Calc_my_IK_node]: tB2G : x=-0.039084 y=-0.575865 z=-0.188168
[INFO] [1794501104.144012297] [Calc_my_IK_node]: IK deg: j0=78.3 j1=-66.7 j2=95.2
[INFO] [0000000000.000000000] [               ]: IK deg: j3=114.9 j4=63.4 j5=57.5
[INFO] [1794501104.144020596] [Calc_my_IK_node]: IK rad: j0=1.366 j1=-1.164 j2=1.662
[INFO] [0000000000.000000000] [               ]: IK rad: j3=2.006 j4=1.106 j5=1.003


[INFO] [1794501105.243852170] [Calc_my_IK_node]: tB2G : x=-0.039084 y=-0.575865 z=-0.188168
[INFO] [1794501105.243895862] [Calc_my_IK_node]: IK deg: j0=78.3 j1=-66.7 j2=95.2
[INFO] [0000000000.000000000] [               ]: IK deg: j3=114.9 j4=63.4 j5=57.5
[INFO] [1794501105.243902669] [Calc_my_IK_node]: IK rad: j0=1.366 j1=-1.164 j2=1.662
[INFO] [0000000000.000000000] [               ]: IK rad: j3=2.006 j4=1.106 j5=1.003


[INFO] [1794501106.243875890] [Calc_my_IK_node]: tB2G : x=-0.039084 y=-0.575865 z=-0.188168
[INFO] [1794501106.243922599] [Calc_my_IK_node]: IK deg: j0=78.3 j1=-66.7 j2=95.2
[INFO] [0000000000.000000000] [               ]: IK deg: j3=114.9 j4=63.4 j5=57.5
[INFO] [1794501106.243928804] [Calc_my_IK_node]: IK rad: j0=1.366 j1=-1.164 j2=1.662
[INFO] [0000000000.000000000] [               ]: IK rad: j3=2.006 j4=1.106 j5=1.003
```

arm_joint6 1.000
left_wheel_joint 0.00
right_wheel_joint 0.00
fl_joint 0.000
fr_joint 0.000
bl_joint 0.000
br_joint 0.000
arm_joint1 1.374
arm_joint2 1.160
arm_joint3 1.660
arm_joint4 2.000
arm_joint5 1.103
gripper_motor_ljoint 0.000
gripper_motor_rjoint 0.000
ct_joint1_2 0.642
ct_joint1_1 0.00
ct_joint2_1 0.00
ct_joint2_2 0.00

ROS Time: 1794501109.79
ROS Elapsed: 1471.05
Wall Time: 1794501109.82
Wall Elapsed: 1471.05
☐ Experimental

그림 1: (영상 1 00:34 부근) 영상 1 의 첫번째 시연이다

영상 1 의 첫번째 시연에서는 TF 를 이용해 엔드포인트의 좌표와 회전정도를 받아와서 내가 계산한 IK 식 들을 이용해 joint0~5 까지 계산한것을 보여준다. rviz 에 조인트 값을 넣을 수 있는 파라미터 gui 와 터미널의 계산 결과가 거의 일치함을 볼 수 있다.

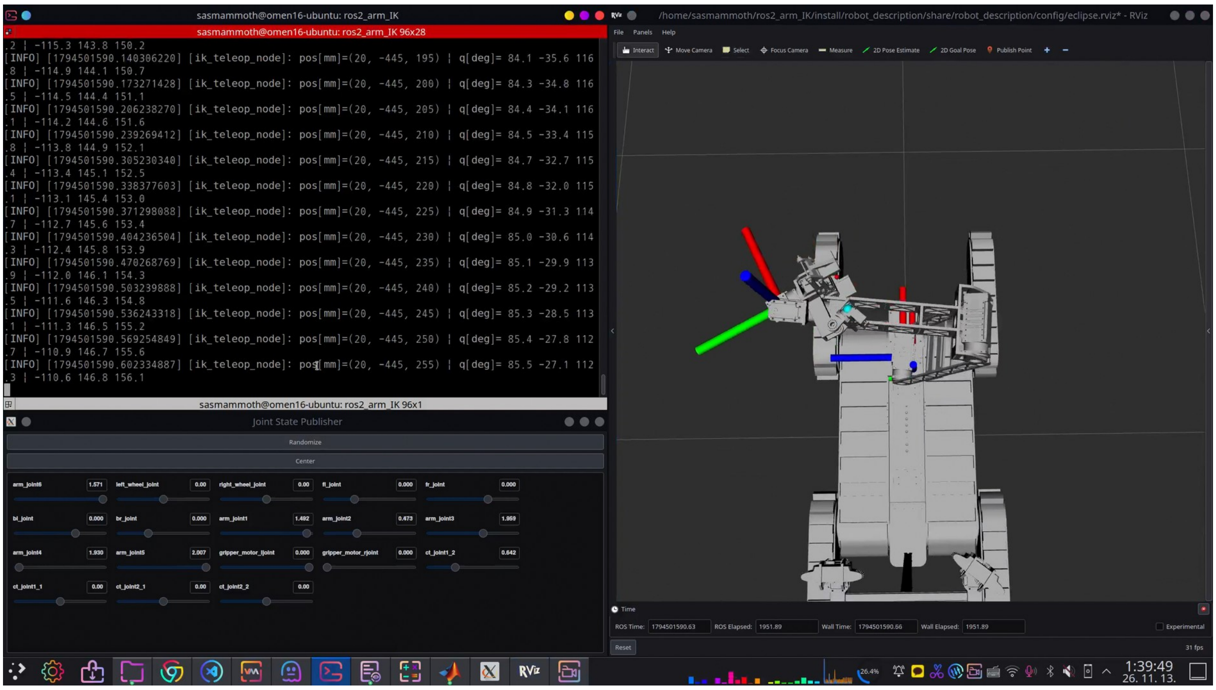

sasmammoth@omen16-ubuntu: ros2_arm_IK
sasmammoth@omen16-ubuntu: ros2_arm_IK 96x28
sasmammoth@omen16-ubuntu: ros2_arm_IK 96x1
Joint State Publisher
Randomize
Center
/home/sasmammoth/ros2_arm_IK/install/robot_description/share/robot_description/config/eclipse.rviz* - RViz
1:33:09
26. 11. 13.

```
[INFO] [0000000000.000000000] [                  ]: IK rad: j3=-2.370 j4=1.073 j5=2.130


[INFO] [1794501165.843837790] [Calc_my_IK_node]: tB2G : x=0.475588 y=-0.657230 z=0.185774
[INFO] [1794501165.843893805] [Calc_my_IK_node]: IK deg: j0=21.3 j1=25.0 j2=25.5
[INFO] [0000000000.000000000] [                  ]: IK deg: j3=-135.8 j4=61.5 j5=122.0
[INFO] [1794501165.843910086] [Calc_my_IK_node]: IK rad: j0=0.372 j1=0.436 j2=0.444
[INFO] [0000000000.000000000] [                  ]: IK rad: j3=-2.370 j4=1.073 j5=2.130


^C[INFO] [1794501166.499465603] [rclcpp]: signal_handler(SIGINT/SIGTERM)
sasmammoth@omen16-ubuntu:ros2_arm_IK$ ros2 run tf_listener
calc_my_IK_node  ik_demo_node     ik_teleop_node
sasmammoth@omen16-ubuntu:ros2_arm_IK$ ros2 run tf_listener ik_demo_node
[INFO] [1794501185.939588648] [ik_demo_node]: 현재 엔드포인트 읽는 중... (/ik/joint_states 로 발
행)
[WARN] [1794501185.959676969] [ik_demo_node]: /joint_states 와 TF 대기 중...
[INFO] [1794501186.900333904] [ik_demo_node]:
  저장한 엔드포인트 pos[mm]=(475.6, -657.2, 185.8) quat=(0.297, 0.090, 0.130, 0.942)
  그때 관절[deg]      =   21.47   25.56   24.55   44.28  -61.31  -58.30
[INFO] [1794501186.900396721] [ik_demo_node]: 원점으로 이동 →2.0 초 대기
[INFO] [1794501188.919749928] [ik_demo_node]: IK 결과로 이동
[INFO] [1794501189.439674417] [ik_demo_node]:
  ===== 결과 =====
  원래 관절[deg] =   21.47   25.56   24.55   44.28  -61.31  -58.30
  IK   관절[deg] =   21.34   24.99   25.45 -135.80   61.48  122.05
  엔드포인트 오차 : 위치 2.296 mm, 회전 0.002 deg
```

arm_joint6 1.571
left_wheel_joint 0.00
right_wheel_joint 0.00
fl_joint 0.000
fr_joint 0.000
bl_joint 0.000
br_joint 0.000
arm_joint1 0.372
arm_joint2 0.436
arm_joint3 0.444
arm_joint4 2.000
arm_joint5 1.073
gripper_motor_ljoint 0.000
gripper_motor_rjoint 0.000
ct_joint1_2 0.642
ct_joint1_1 0.00
ct_joint2_1 0.00
ct_joint2_2 0.00

그림 2: (영상 1 01:53 부근) 영상 1의 두번 째 시연이다

영상 1의 2번째 시연은 AI를 이용해 제작한 시연이다. 엔드포인트의 위치와 자세를 기억한 뒤, 모든 조인트를 원점으로 되돌리고 2초 후에 IK를 이용하여 조인트 6개의 각을 계산하여 처음과의 차이를 구하는 시연이다. 어느정도 초기와 가깝게 나온 모습니다.


sasmammoth@omen16-ubuntu: ros2_arm_IK
sasmammoth@omen16-ubuntu: ros2_arm_IK 96x28
sasmammoth@omen16-ubuntu: ros2_arm_IK 96x1
Joint State Publisher
Randomize
Center
/home/sasmammoth/ros2_arm_IK/install/robot_description/share/robot_description/config/eclipse.rviz* - RViz
1:39:49
26. 11. 13.

```
.2 ¦ -115.3 143.8 150.2
[INFO] [1794501590.140306220] [ik_teleop_node]: pos[mm]=(20, -445, 195) ¦ q[deg]= 84.1 -35.6 116
.8 ¦ -114.9 144.1 150.7
[INFO] [1794501590.173271428] [ik_teleop_node]: pos[mm]=(20, -445, 200) ¦ q[deg]= 84.3 -34.8 116
.5 ¦ -114.5 144.4 151.1
[INFO] [1794501590.206238270] [ik_teleop_node]: pos[mm]=(20, -445, 205) ¦ q[deg]= 84.4 -34.1 116
.1 ¦ -114.2 144.6 151.6
[INFO] [1794501590.239269412] [ik_teleop_node]: pos[mm]=(20, -445, 210) ¦ q[deg]= 84.5 -33.4 115
.8 ¦ -113.8 144.9 152.1
[INFO] [1794501590.305230340] [ik_teleop_node]: pos[mm]=(20, -445, 215) ¦ q[deg]= 84.7 -32.7 115
.4 ¦ -113.4 145.1 152.5
[INFO] [1794501590.338377603] [ik_teleop_node]: pos[mm]=(20, -445, 220) ¦ q[deg]= 84.8 -32.0 115
.1 ¦ -113.1 145.4 153.0
[INFO] [1794501590.371298088] [ik_teleop_node]: pos[mm]=(20, -445, 225) ¦ q[deg]= 84.9 -31.3 114
.7 ¦ -112.7 145.6 153.4
[INFO] [1794501590.404236504] [ik_teleop_node]: pos[mm]=(20, -445, 230) ¦ q[deg]= 85.0 -30.6 114
.3 ¦ -112.4 145.8 153.9
[INFO] [1794501590.470268769] [ik_teleop_node]: pos[mm]=(20, -445, 235) ¦ q[deg]= 85.1 -29.9 113
.9 ¦ -112.0 146.1 154.3
[INFO] [1794501590.503239888] [ik_teleop_node]: pos[mm]=(20, -445, 240) ¦ q[deg]= 85.2 -29.2 113
.5 ¦ -111.6 146.3 154.8
[INFO] [1794501590.536243318] [ik_teleop_node]: pos[mm]=(20, -445, 245) ¦ q[deg]= 85.3 -28.5 113
.1 ¦ -111.3 146.5 155.2
[INFO] [1794501590.569254849] [ik_teleop_node]: pos[mm]=(20, -445, 250) ¦ q[deg]= 85.4 -27.8 112
.7 ¦ -110.9 146.7 155.6
[INFO] [1794501590.602334887] [ik_teleop_node]: pos[mm]=(20, -445, 255) ¦ q[deg]= 85.5 -27.1 112
.3 ¦ -110.6 146.8 156.1
```

arm_joint6 1.571
left_wheel_joint 0.00
right_wheel_joint 0.00
fl_joint 0.000
fr_joint 0.000
bl_joint 0.000
br_joint 0.000
arm_joint1 1.492
arm_joint2 0.473
arm_joint3 1.959
arm_joint4 1.930
arm_joint5 2.007
gripper_motor_ljoint 0.000
gripper_motor_rjoint 0.000
ct_joint1_2 0.642
ct_joint1_1 0.00
ct_joint2_1 0.00
ct_joint2_2 0.00

그림 3: 영상 2 00:40 부근

영상 2는 키보드 조작을 이용해 로봇의 베이스 좌표의 xyz, pry를 기반으로 엔드포인트를 움직이는 시연이다. IK 계산은 영상 1에서 사용한 것을 그대로 사용하였고, 기타 제반사항을 AI를 이용해 제작하였다.

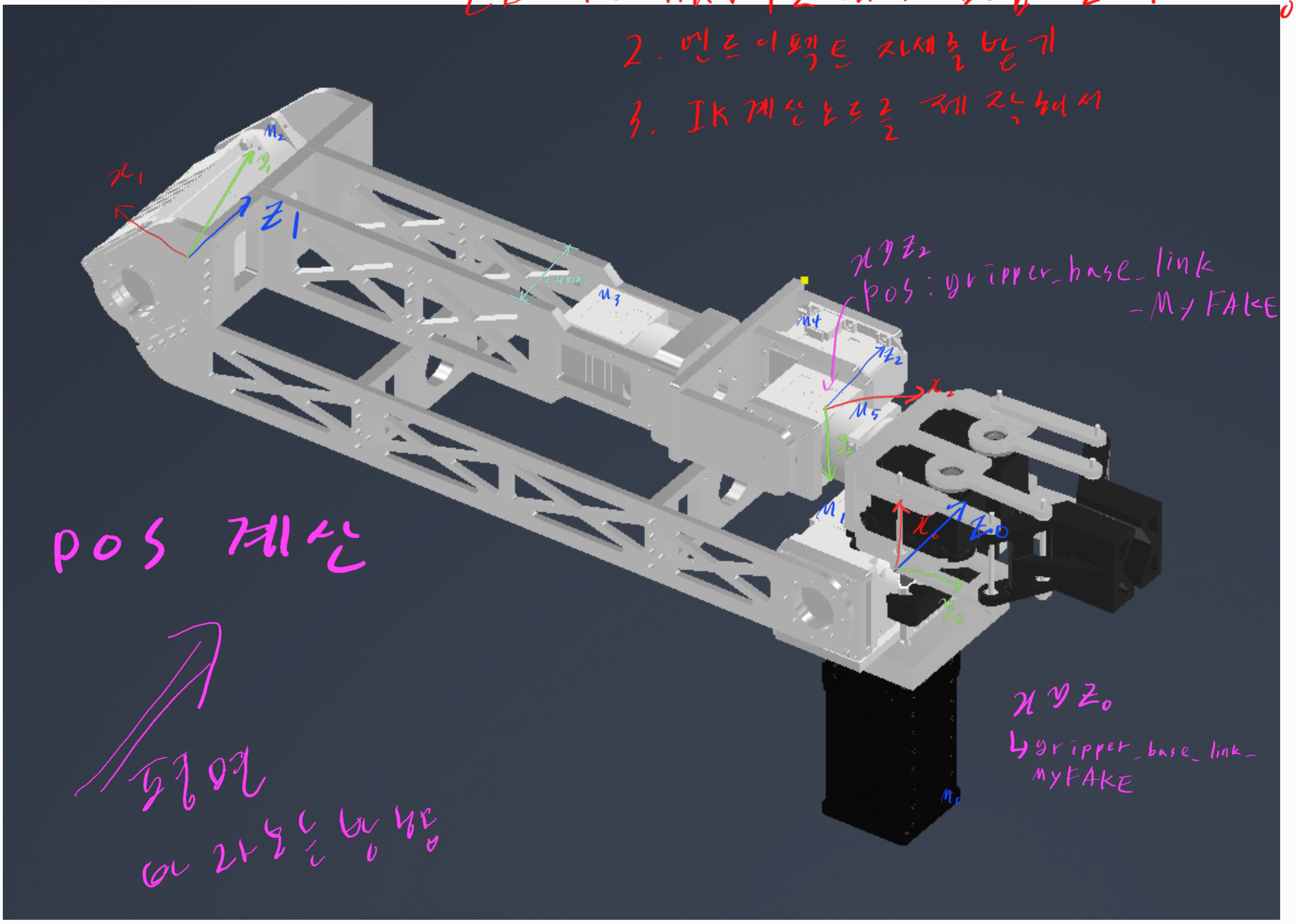
2. IK 계산 및 기타

IK를 편히 계산하기 위하여 원본 robot_description에서 몇가지 링크를 추가하였으며 이는 TF_Frame.pdf에 나와있다.

과제: 1. Rviz 에서 마음대로 위치 이동

2. 엔드이펙트 자세를 받기

3. IK 계산 노드를 제작해서



1. 좌표를 평면으로 잘라서 2링크로 해석. (M_1, M_2 계산)

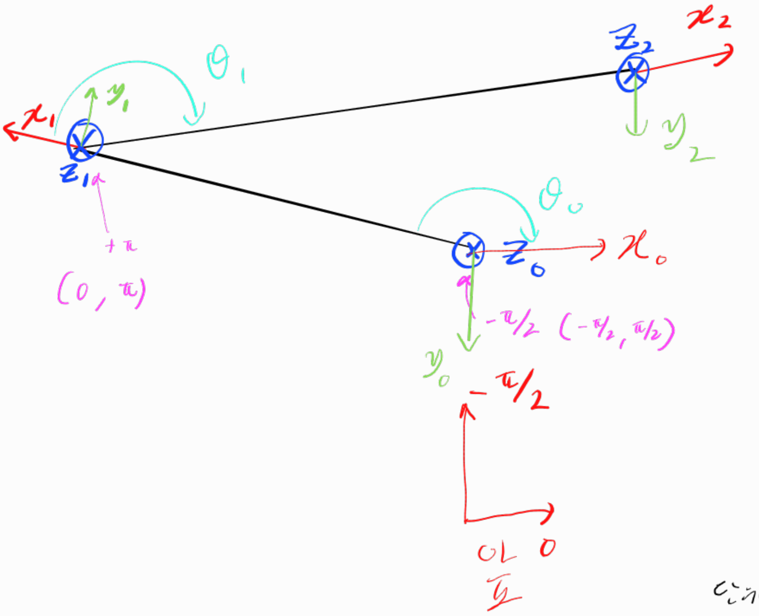
2. 좌표평면 tan 으로 M_0 구하기

3. 회전행렬과 Pitch-Roll-Yaw, 역행렬(전치행렬)을 이용해
   M_{345} 구하기

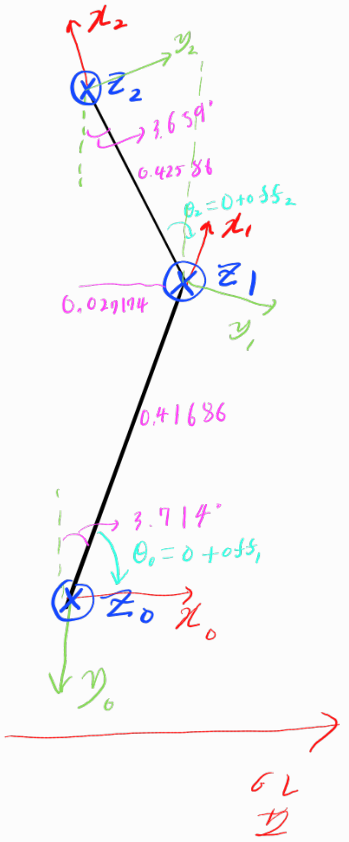
4. 필요에따라 Rviz 의 .xacro를 조작하여 링크를 추가한다.

★ 인벤터랑 RVIZ 간 수치가 일부 달라서
RVIZ로 수정 계산 함

접힌 로봇팔 (RVIZ)



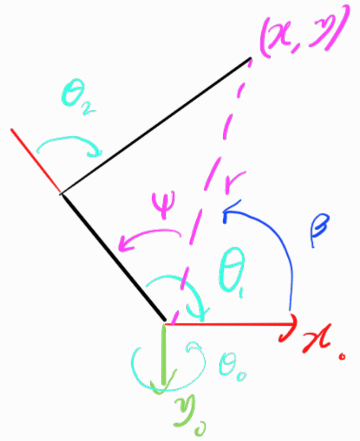
RVIZ 원점



단위 : m, deg

	d_i	θ_i	a_i	α_i
1	0	$0 - 86.286°$	0.41686	0
2	0	$0 - 7.373°$	0.42586	0

포지션 계산 : 코드(2)

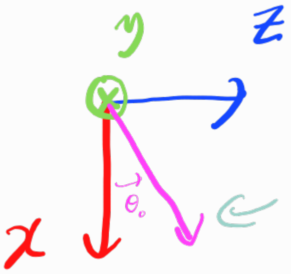


$$C_2 = \frac{x^2 + y^2 - a_1^2 - a_2^2}{2 a_1 a_2}$$

$$S_2 = \pm\sqrt{1 - C_2^2}$$

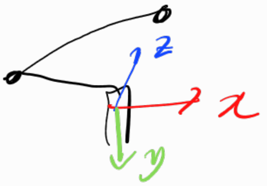
$$\theta_2 = \operatorname{atan2}(S_2, C_2)$$

$$\theta_1 = \operatorname{atan2}(y, x) - \operatorname{atan2}(a_2 S_2 + a_1 + a_2 C_2)$$

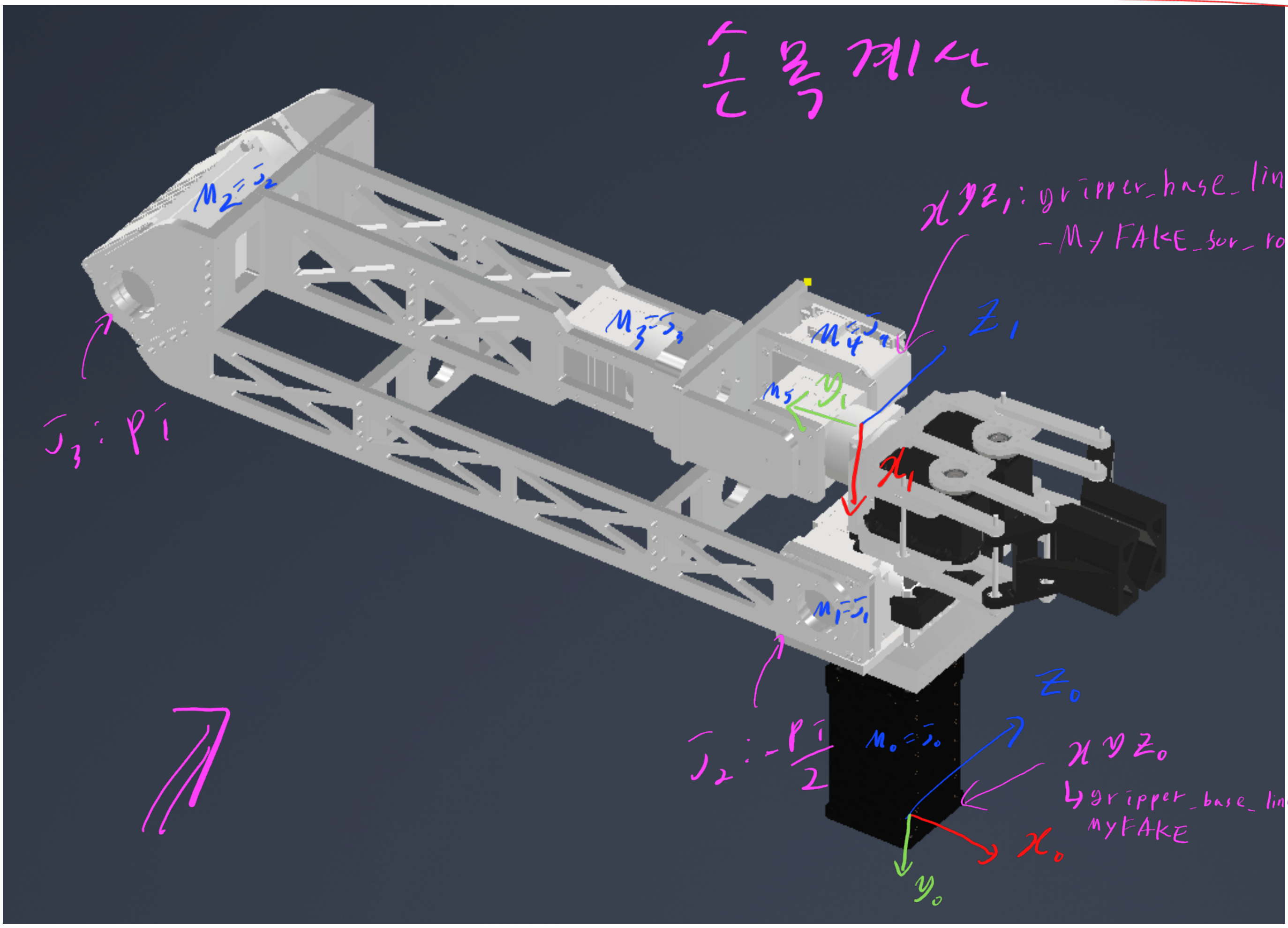


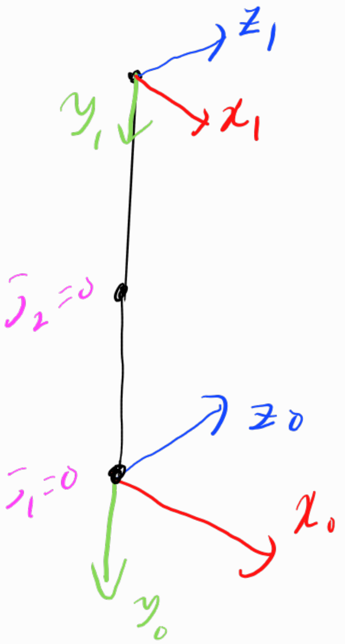
$\theta_0 = \text{atan2}(z, x)$

← 동평면 기준 : 곤등 ①



손목 계산





$$R_{\text{목표}} = R_{\text{팔}} \cdot R_{\text{손목}}$$

$$R_{\text{손목}} = R_{\text{팔}}^{-1} \cdot R_{\text{목표}}$$

$$\rightarrow R_{\text{팔}}^{T}$$

$$R_{\text{팔}} = R_{J_0(M_0)} \circ R_{J_1(M_1)} \circ R_{J_2(M_2)}$$

$$= [-\text{Pitch}(J_0)] \circ [+\text{Yow}(J_1)] \circ [+\text{Yow}(J_2)]$$

$$R_{손목} = [-P(\bar{J}_3)]\cdot[\,Y(\bar{J}_4)]\cdot[-P(\bar{J}_5)]$$

$-\bar{J}_3 = a$, $\bar{J}_4 = b$, $-\bar{J}_5 = c$

$$= \begin{bmatrix} ca & 0 & sa \\ 0 & 1 & 0 \\ -sa & 0 & ca \end{bmatrix} \cdot \begin{bmatrix} cb & -sb & 0 \\ sb & cb & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} cc & 0 & sc \\ 0 & 1 & 0 \\ -sc & 0 & cc \end{bmatrix}$$

$$= \begin{bmatrix} ca\cdot cb\cdot cc - sa\cdot sc & \boxed{-ca\cdot sb} & ca\,cb\,sc\cdot sa\,cc \\ \boxed{sb\cdot cc} & \boxed{cb} & \boxed{sb\cdot sc} \\ -sa\cdot cb\cdot cc - ca\cdot sc & \boxed{sa\cdot sb} & -sa\cdot cb\cdot sc + ca\cdot cc \end{bmatrix}$$

b, c b a, b

이거 사용

$$\bar{J}_4 = b = \operatorname{atan2}\left(\pm\sqrt{R_{12}^2 + R_{32}^2},\ R_{22}\right)$$

$$\bar{J}_3 = -a = -\operatorname{atan2}(R_{32}, -R_{12})$$

$$\bar{J}_5 = -c = -\operatorname{atan2}(R_{23}, R_{21})$$